

ENTREGA QUINTA SESIÓN

APROXIMACIÓN 1



Nombre	NIA	Correo	Grupo	Equipo
Jorge López Alonso	100495876	100495876@alumnos.uc3m.es	85	509
Raoul Vlad Ivaszuk Ivaszuk	100508621	100508621@alumnos.uc3m.es	85	509



ÍNDICE

1. Gramática Inicial	1
2. Gramática Extendida	2
3. Parser	4
4. Diagrama Sintáctico	5
5. Pruebas y Evaluaciones	6

1. Gramática Inicial

En primer lugar, tras leer el código inicial proporcionado, hemos detectado que `red_lex()` nos proporciona los siguientes elementos léxicos:

- `T_NUMERO`: es el token correspondiente a los números.
- `T_VARIABLE`: es el token correspondiente a las variables.
- `T_OPERADOR`: es el token correspondiente a los operadores.
- Los literales que posteriormente usaremos como pueden ser '=' o '?'.

Tras este análisis, diseñamos esta gramática inicial, siguiendo las especificaciones iniciales:

```
None
Línea -> Expresión \n
Expresión -> (operador Parametro Parametro) | Número
Parámetro -> Expresión | Número
Operador -> + | - | * | /
Número -> 0..9
```

El problema es que al introducir la gramática en JFLAP, nos dice que no es de tipo LL(1), ya que se produce un conflicto FIRST-FIRST, lo que provoca una ambigüedad. Para solventar este conflicto hemos rediseñado la gramática, quitando la posibilidad de que parámetro produzca numero ($P \rightarrow N$). La gramática inicial queda de la siguiente manera:

Table Text Size

Table Text Size

L	→	E						
E	→	(OPP)	E	{ (, 9 }	FIRST	{ \$, (, 9,) }	FOLLOW	
E	→	N	L	{ (, 9 }		{ \$ }		
P	→	E	N	{ 9 }		{ \$, (, 9,) }		
O	→	o	O	{ o }		{ (, 9 }		
N	→	9	P	{ (, 9 }		{ (, 9,) }		
				(	)	9	o	\$
E	(OPP)		N					
L	E		E					
N			9					
O						o		
P	E		E					

2. Gramática Extendida

A continuación, hemos formulado lo que sería la gramática extendida, siguiendo las especificaciones que se nos indican en el enunciado. Para ello, ya que volvíamos a obtener un conflicto primero-primero, volvemos a hacer un pequeño rediseño de esta y añadimos el no terminal Resto, además también hemos quitado la redenominación de expresión como parámetro. Con estos cambios también permitimos las asignaciones encadenadas. El resultado final que obtuvimos es:

Variable -> a..z

En JFlap tuvimos el siguiente resultado:

Table Text Size			Table Text Size							
L	→	E								
E	→	(R)								
E	→	N								
E	→	V								
R	→	OEE								
R	→	=VE								
O	→	o								
N	→	9								
V	→	a								

			FIRST				FOLLOW			
E			{ a, (, 9 }				{ a, \$, (, 9,) }			
L			{ a, (, 9 }				{ \$ }			
N			{ 9 }				{ a, \$, (, 9,) }			
O			{ o }				{ a, (, 9 }			
R			{ =, o }				{) }			
V			{ a }				{ a, \$, (, 9,) }			

	(	)	9	=	a	o	\$
E	(R)		N		V		
L	E		E		E		
N			9				
O						o	
R				=VE		OEE	
V					a		

Posteriormente, para cumplir la especificación de asignaciones condicionales, hemos decidido añadir un no terminal C (Condicional), para poder cumplir con dicha directiva. Este nuevo símbolo nos permite tener la posibilidad de que la asignación tenga 1 sola expresión,

para la asignación normal, o tres expresiones, para la asignación ternaria. Además hemos permitido que Resto reconozca el operador ternario ? y sus tres expresiones. Así obtenemos nuestra gramática final:

```
None
Linea -> Expresión \n
Expresión -> (Resto) | Número | Variable
Resto -> Operador Expresión Expresión | = Variable Expresión Condición |
? Expresión Expresión Expresión
Condición-> Lambda | Expresión Expresión
Operador -> + | - | * | /
Número -> 0..9
Variable -> a..z
```

En JFlap obtenemos los siguientes conjuntos Primero y Siguiente:

Table Text Size		
L	→	E
E	→	(R)
E	→	N
E	→	V
R	→	OEE
R	→	=VEC
R	→	?EEE
C	→	λ
C	→	EE
O	→	o
N	→	9
V	→	a

3. Parser

El analizador sintáctico que hemos desarrollado sigue muy de cerca la gramática que hemos definido en el punto anterior. Hemos definido funciones para cada símbolo no terminal que tenemos en la gramática donde cada una se encarga de analizar sus producciones y llamar de forma recursiva a los analizadores de los símbolos no terminales que estas producen.

Tenemos la función `ParserAxiom()` que se encarga de analizar la producción del NT Línea, el axioma de la gramática. Todos los demás analizadores siguen el siguiente patrón de nombramiento: `ParserX()` donde X es el nombre del NT que analiza.

En este diseño tenemos tres funciones que consume o analizan los tokens obtenidos por el analizador lexico, estas son `ParseNumero()`, `ParseVariable`, `ParseOperador()`, las dos primeras, para cumplir con el objetivo de la traducción, imprimen el valor de su token correspondiente, la última no.

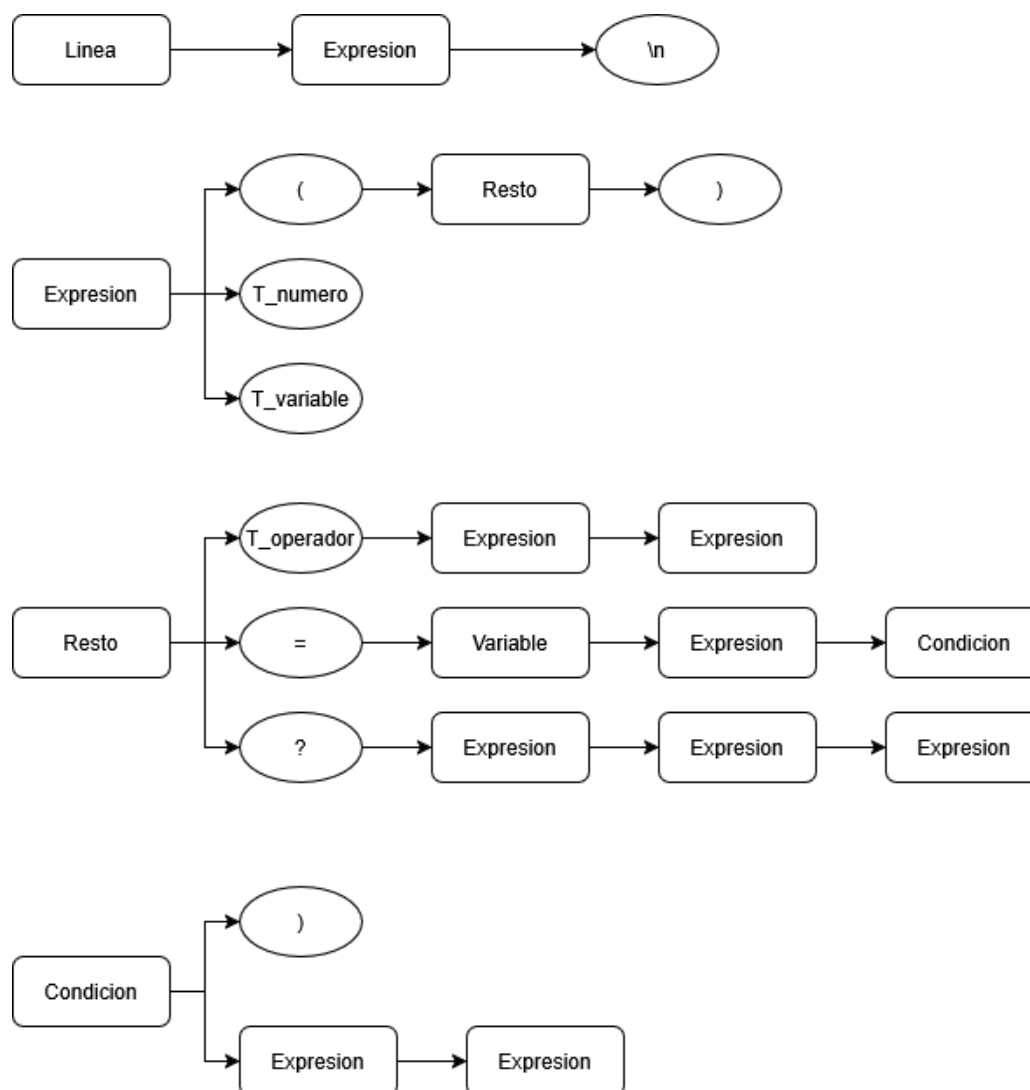
El siguiente analizador que se ha desarrollado es el encargado del NT Condición, aquí nos encontramos con el problema de que este produce lambda, por ello hemos tenido que hacer uso de la tabla de los conjuntos primero y siguiente para poder distinguir los posibles casos que se producen. En el caso de no producir lambda, se encarga de hacer una traducción de la entrada a lo correspondiente a las dos últimas expresiones de una asignación condicional ternaria.

En el caso del analizador de Resto, se distinguen tres casos, que se corresponden con cada uno de los tres posibles tokens que componen el conjunto primero de este NT. Si la entrada se corresponde con un operador, lo primero que se hace es guardar el valor de este para posterior poder usarlo, tras consumirlo, para dar el formato correspondiente de salida a esta posible entrada. Cuando la entrada se corresponde con el operador de condición, este guarda el nombre de la variable, para que de forma similar al caso anterior, se pueda formatear la traducción una vez consumidos los tokens. Por último cuando la entrada se corresponde con una condición ternaria, este nuevamente formatea la salida en caso de que la entrada sea correcta.

A continuación, el analizador del NT Expresión, nuevamente diferencia sus producciones en función de los tokens que pertenecen a su conjunto primero, ya sea para detectar un número, una variable o una expresión más compleja.

Por último, el analizador del axioma es el que se encarga de leer las líneas que entran en la consola y comenzar con el proceso recursivo de análisis que hemos definido hasta ahora, cabe destacar que es necesario este analizador para poder detectar los saltos de línea de una forma más cómoda y correcta.

4. Diagrama Sintáctico



En este diagrama se han obviado los analizadores que reconocen los tokens del analizador léxico siendo estos representados como tokens y por ello como símbolos no terminales y en condición, la primera producción se ha representado como el conjunto siguiente de condición en lugar de lambda, de la misma forma que en el código.

5. Pruebas y Evaluaciones

Las pruebas que hemos realizado pretenden abarcar desde los casos más sencillos como el simple reconocimiento de un número el nombre de una variable hasta expresiones más complejas que combinan asignaciones con condiciones ternarias con operaciones. En todos los casos que hemos probado las soluciones y han sido las esperadas por ello podemos concluir que nuestro analizador sintáctico reconoce y traduce la gramática correctamente.

None

```
5 = 5
x = x
A1 = A1
(+ 2 3) = (2 + 3)
(- 10 4) = (10 - 4)
(* A 5) = (A * 5)
(/ 100 10) = (100 / 10)
(+ (* 2 3) 4) = ((2 * 3) + 4)
(- 10 (/ 8 2)) = (10 - (8 / 2))
(* (+ 1 2)(- 5 3)) = ((1 + 2) * (5 - 3))
(+ (+ (+ 1 2) 3) 4) = (((1 + 2) + 3) + 4)
(+ 1 (+ 2 (+ 3 4))) = (1 + (2 + (3 + 4)))
(= x 5) = (x = 5)
(= va (+ 10 20)) = (va = (10 + 20))
(= y 10 1 0) = (y = 10 ? 1 : 0)
(= z (* 2 5) x y) = (z = (2 * 5) ? x : y)
(= a 1 (= b 2)(= c 3)) = (a = 1 ? (b = 2) : (c = 3))
(? 1 2 3) = (1 ? 2 : 3)
(? x (+ 1 2)(- 3 4)) = (x ? (1 + 2) : (3 - 4))
(? (- 5 5) 10 20) = ((5 - 5) ? 10 : 20)
(? 1 (? 2 3 4) 5) = (1 ? (2 ? 3 : 4) : 5)
(? 0 1 (? 0 2 3)) = (0 ? 1 : (0 ? 2 : 3))
(+ (= x 5)(? y 10 20)) = ((x = 5) + (y ? 10 : 20))
(= re (? 1 2 3) 4 5) = (re = (1 ? 2 : 3) ? 4 : 5)
(? (= fl 1)(* 10 2)(/ 10 2)) = ((fl = 1) ? (10 * 2) : (10 / 2))
(* (= a (+ 1 2))(= b (? 1 0 1))) = ((a = (1 + 2)) * (b = (1 ? 0 : 1)))
```